程序设计

讲义

2026 年秋季

毕朝日

复旦大学集成电路与微纳电子创新学院

zhaori_bi@fudan.edu.cn https://bzr2915.github.io/bzr2915/

目录

第五讲　结构化控制：循环与分支跳转 **4**

5.1　把执行过程画出来 4

5.2　while：读到一个数，就处理一个数 5

5.2.1　条件先判断，循环体后执行 5

5.2.2　把读入动作放进条件 7

5.2.3　循环为什么能够结束 7

5.2.4　花括号与空语句决定循环体的边界 8

5.3　for：把初值、条件与更新放在一起 9

5.3.1　一张立方表的执行顺序 9

5.3.2　更新不一定是加 1 11

5.3.3　省略初始化、条件或更新 12

5.3.4　用逗号表达式同时更新两项状态 13

5.4　do-while：先做一次，再决定是否继续 13

5.5　嵌套循环：一行怎样变成一张表 15

5.6　if 与 else：根据条件选择处理方式 16

5.6.1　零度以下的日子有多少 16

5.6.2　多个独立 if 与一条分支链 18

5.6.3　else 究竟与哪一个 if 配对 19

5.6.4　阶梯电费为什么只对超出的部分加价 20

5.7　把多个条件组合起来 21

5.7.1　逻辑与、逻辑或、逻辑非 21

5.7.2　优先级规定分组，短路规定是否继续求值 22

5.7.3　读字符时，先存入，再判断 23

5.7.4　句号前，除了引号还剩多少字符 24

5.8　条件运算符：从两个值中选一个 25

5.9　把循环和分支合起来：约数与素数 27

5.10　continue：这条记录不处理，接着读下一条 28

5.10.1　成绩不合范围时，跳过统计 28

5.10.2　continue 之后，for 与 while 去往不同位置 30

5.11 break：结束当前循环 32
5.11.1 长读到了，宽却没有读到 32
5.11.2 跳出一层，不等于跳出全部层次 33
5.12 switch：在若干离散值中选入口 34
5.12.1 case 标签不是一条条独立的 if 34
5.12.2 没有 break 就继续向下执行 36
5.13 综合例题：先确定状态，再安排控制流 37
5.13.1 例题一：三个整数中的最大数 37
5.13.2 例题二：三个整数从大到小排列 37
5.13.3 例题三：二次方程求根与退化情况 38
5.13.4 例题四：一行字符分成三类 40
5.13.5 例题五：用牛顿法逐次逼近方程的根 41
5.13.6 例题六：余数条件如何减少查找次数 43
5.13.7 例题七：最大值第一次出现在第几项 44
5.13.8 例题八：50 张纸币怎样凑成 116 元 45
5.13.9 例题九：边长不超过 25 的整数直角三角形 45
5.13.10 例题十：三个人都只猜对一半 46
5.13.11 例题十一：方程的所有正整数解 47
5.14 习题与解析 48
5.15 检查循环与分支 52

第五讲　结构化控制：循环与分支跳转

实验室已经记录了一段时间内每天的最低温度，现在需要根据这些记录，输出总天数，以及最低温度低于 0 摄氏度的天数占总天数的百分比。每个温度值对应一天，程序只需读入这些数值，不需要读入日期。

输入时，依次键入温度值，用空格或换行分隔，最后输入字母 `q` 表示结束，不必预先告诉程序共有多少条记录。例如，输入 `5 -2 0 -3 q`，表示四天的最低温度分别为 5、−2、0 和 −3 摄氏度。其中两天低于 0 摄氏度，占 $2/4 \times 100\% = 50\%$；恰好为 0 摄氏度的那一天不计入低温天数。

程序可以用两个计数器分别保存总天数和低温天数，初值都设为 0。每成功读入一个温度，总天数就加 1；如果这个温度低于 0 摄氏度，低温天数也加 1。处理完一条记录后，再读取下一条，直到不能继续读入温度。让这组操作反复执行，需要**循环**；根据温度决定是否增加低温天数，需要**分支**。

停止读取后，还要判断总天数是否为 0。有记录时才能计算百分比；如果一开始就输入 `q`，程序应提示“没有数据”，避免用 0 作除数。[1]

5.1　把执行过程画出来

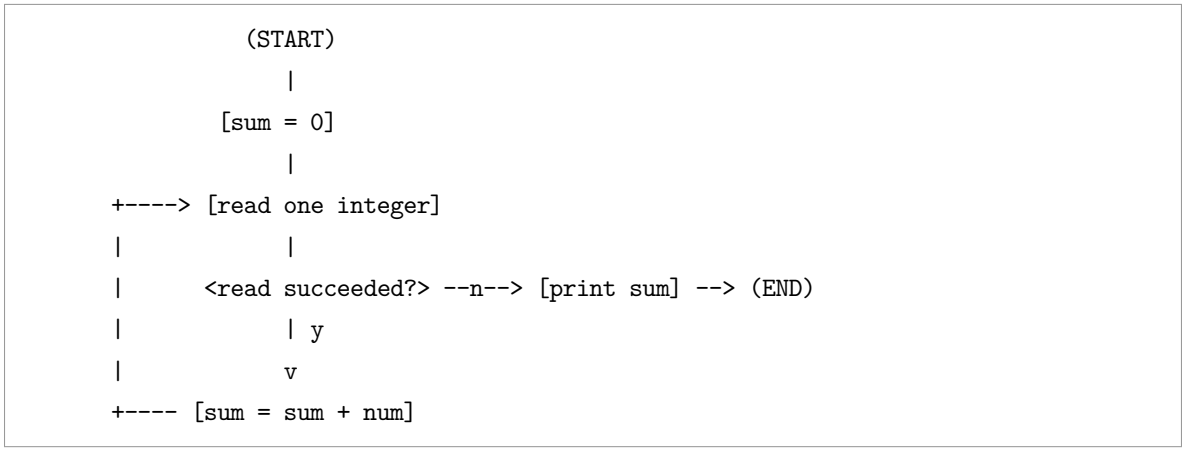
一段程序实际执行的先后次序称为**控制流**。顺序结构让语句依次执行；选择结构依据条件执行某个分支；循环结构让一段操作重复执行。把这些结构组合、嵌套起来，就能让程序文本清楚地反映处理过程。

流程图用图形描述控制流：圆角框表示开始或结束，矩形表示处理，菱形表示条件判断，箭头表示下一步。判断出口标注 `y` 和 `n`，分别表示真与假。**伪代码**则用接近自然语言的步骤描述同一过程，不要求符合 C 语法。

为了看清重复读取的过程，先考虑一个更简单的任务：依次读入整数，读到不能转换为整数的输入时停止，输出已读入整数的和。例如，输入 45、78、94，再输入字母 q，结果应为 217。

下面用 ASCII 字符画出这一流程。图中的 `sum` 是累计和，`num` 是本次输入值；菱形位置以尖括号表示。图中的 `read one integer` 表示尝试读入一个整数，`read succeeded?` 判断本次读取是否成功，`print sum` 表示输出累计和。每次沿左侧箭头返回，都要先尝试读取下一个整数，再判断是否继续。

[1] 代码按 C11 编译。数值输入为有限值，输入值、中间运算结果及计数均须在相应类型的表示范围内。



第一次读取之前，累计和为 0。每次读取成功，就把新读到的整数加到累计和中，然后继续读取；读取失败时，不再累加，沿右侧路径输出结果。若一次也没有读到整数，输出的和就是 0。

历史旁注

1968 年，Edsger W. Dijkstra 在关于 `go to` 的文章中讨论了一个阅读程序的困难：文本静止地写在纸上，执行过程却随时间变化。程序员必须知道执行到了哪里，才能解释一个变量此刻的意义。例如人数计数器在“有人进入房间”和“计数器加 1”之间，还没有反映最新人数。限制任意跳转、使用清楚的分支和循环，有助于对应文本位置与执行阶段。（原文）

5.2 while：读到一个数，就处理一个数

5.2.1 条件先判断，循环体后执行

`while` 的常用形式如下。花括号括住的部分称为**循环体**，其中写需要重复执行的语句。

```
while (condition) {
    /* loop body */
}
```

这里的 `condition` 代表条件表达式，不是 C 的关键字；`/* loop body */`是注释，提示应在此处填入实际的循环体。本讲的语法通式只表示语句结构，不能作为完整程序直接编译运行。

花括号把其中的多条语句组成一条**复合语句**。若循环体只有一条语句，可以用这条语句整体替换花括号及其内容；保留花括号则更容易看清循环体的边界。条件后的右圆括号与左花括号之间不加分号，结束花括号后也不需要分号；循环体内部的赋值、函数调用等语句仍按各自的语法以分号结尾。

执行 while 时，先计算条件表达式的值：非零表示真，进入循环体；0 表示假，跳过循环体，继续执行循环后的语句。循环体执行完毕，再回到条件处重新判断。因此，第一次判断就为假时，循环体一次也不执行。

下面的程序实现整数求和。num 保存本次读到的整数，sum 保存累计和，status 保存 scanf 的返回值。本次 scanf 只要求读入一个整数，返回 1 就表示已成功给 num 赋值。0L 是 long 类型的整数常量，输入和输出中的%ld 分别与 long 对象的地址和值相配。

```
#include <stdio.h>

int main(void) {
    long num, sum = 0L;
    int status;

    printf("Enter integers; q to stop.\n");
    status = scanf("%ld", &num);
    while (status == 1) {
        sum += num;
        status = scanf("%ld", &num);
    }
    printf("sum = %ld\n", sum);
    return 0;
}
```

输入 45 78 94 q 并按回车，最后输出 sum = 217。键盘回显的输入文字不属于程序输出。这里用 q 结束输入，是因为它不能被%ld 转换成整数；程序没有专门识别字母 q，遇到其他不能转换为整数的内容也会停止。仅输入空格或换行不会停止，因为 scanf 会跳过这些空白，继续等待整数。

下表记录每次执行 while (status == 1) 时的状态。此时本轮读取已经结束，但本轮累加尚未执行，因此 sum 只包含此前成功读入的整数。

本次遇到的输入	status	num	sum	接下来执行什么
45	1	45	0	加上 45，再读入
78	1	78	45	加上 78，再读入
94	1	94	123	加上 94，再读入
q	0	94	217	离开循环，输出和

遇到 q 时，整数转换失败，scanf 返回 0，num 仍保留上一次读到的 94；q 没有被读取，仍留在输入流中。因为 status == 1 为假，程序离开循环，不会把 94 再加一次。若开

始就输入 q，num 尚未得到值，但循环体不会执行，也就不会使用 num；初始化为 0 的 sum 直接作为结果输出。

5.2.2 把读入动作放进条件

把 scanf 放进循环条件，每次判断时就会读取一个数，原来的两处输入语句可以合并为一处：

```
#include <stdio.h>

int main(void) {
    long num, sum = 0L;
    while (scanf("%ld", &num) == 1) {
        sum += num;
    }
    printf("sum = %ld\n", sum);
    return 0;
}
```

每次到达循环条件时，先执行 scanf，再把它的返回值与 1 比较。只有成功读入一个整数，才进入循环体，把 num 加到 sum 中；读取失败就直接结束循环。因此，每次参与累加的都是本次成功读入的新值。

不能把条件继续简写成 while (scanf("%ld", &num))。匹配失败时返回 0，确实会停止；但如果在赋值之前遇到文件结束或输入错误，返回的是 EOF，这是一个负的 int 常量，仍然是非零值。直接判断返回值会错误地进入循环体。对这里的一项转换，应明确写 == 1。

检查返回值也不等于验证整行输入。例如输入 12x，第一次转换能成功读到 12，x 留给下一次输入；第二次转换失败，程序停止。检查返回值不能解决输入数值超出目标类型表示范围的问题。

5.2.3 循环为什么能够结束

求和程序在开始运行时不知道用户会输入多少个数，实际循环次数由输入决定，这称为**不定次循环**。程序必须继续读取，才有机会发现输入已经结束。

在使用 status 的求和程序中，若第一次读取成功，却删去了循环体末尾的 scanf，那么 status 会一直保持 1，num 也一直保留第一个整数。程序便会反复累加同一个数，不能通过新的输入决定何时停止。

另一些循环用整数记录已经处理到哪一项，称为**计数循环**。例如累加 1 到 100：

```c
#include <stdio.h>

int main(void) {
    int i = 1, sum = 0;
    while (i <= 100) {
        sum += i;
        i++;
    }
    printf("sum = %d, i = %d\n", sum, i);
    return 0;
}
```

输出为 `sum = 5050, i = 101`。第一次进入循环体时，`i` 为 1，`sum` 为 0，尚未累加任何数；执行 `sum += i` 后，和变成 1，再执行 `i++`，使 `i` 变成 2。以后每轮都按这个顺序处理。

因此，每轮进入循环体时，`sum` 保存的是此前从 1 到 `i-1` 的累加结果。本轮加上 `i` 后，再把 `i` 增加 1，准备处理下一个数。最后一轮加上 100，然后 `i` 变成 101；下一次判断 `i <= 100` 为假，循环结束。101 表示下一个待检查的数，最后实际加到 `sum` 中的数仍是 100。

检查计数循环，要同时检查初值、条件和更新方向。若 `index` 从 1 开始，却写成 `while (--index < 5)`，变量越减越小，并没有朝使条件为假的方向变化。有符号整数减到最小值后再减 1 会导致未定义行为，不能靠“变成最大正数”来解释它的终止。

5.2.4　花括号与空语句决定循环体的边界

下面两段都是**错误示例**，不能按缩进推断它们的行为：

```c
int i = 1, sum = 0;
while (i <= 100)
    sum += i;
    i++;
```

没有花括号时，`while` 只控制紧随其后的那一条语句，即 `sum += i;`。缩进不能把 `i++` 放进循环体；只有离开循环后才会执行它。于是 `i` 一直为 1，条件 `i <= 100` 一直为真，`sum` 反复加 1，最终会发生有符号整数溢出，导致未定义行为。应像前一个完整程序那样，用花括号把累加和自增一起放入循环体。

```
int i = 1, sum = 0;
while (i <= 100);
    sum += i++;
```

紧跟条件的分号构成**空语句**，即不做任何操作的语句。while 把这条空语句当作循环体，后面的 sum += i++; 已经在循环外。按这个控制结构，反复判断条件时不会改变 i，因而无法靠 i++ 推进循环。这里应删除多余的分号，并用花括号明确括住循环体。

空循环体也有合理用途。例如只想不断读入并丢弃整数，直到转换失败，可以写：

```
int num;
while (scanf("%d", &num) == 1)
    ;
```

操作发生在条件里，循环体无需再做事情。分号单独占一行，能表明这是有意的空语句。这个例子丢弃的是一个个整数转换结果；scanf 会跳过前导空白，也能接受正负号，不能把它理解成逐字符丢弃所有数字。

5.3 for：把初值、条件与更新放在一起

5.3.1 一张立方表的执行顺序

打印 1 到 6 的立方，需要从 1 开始，每次加 1，超过 6 时停止。for 把初始化、继续条件和更新操作写在同一对圆括号内，常用形式是：

```
for (initialization; condition; update) {
    /* loop body */
}
```

三个英文名称都是占位符。initialization 表示初始化，可以是表达式，如 num = 1，也可以是变量声明，如 int num = 1；condition 表示继续循环的条件；update 表示每轮结束时执行的更新表达式，如 num++。注释所在的位置填入实际的循环体。

执行顺序是：先执行一次初始化，再判断条件；条件为真时执行循环体，正常到达循环体末尾后执行更新，再回到条件处判断。条件为假时，直接执行循环后的语句，不再执行循环体和更新。初始化只执行一次，第一次判断为假时，循环体一次也不执行。

圆括号中的三项都可以省略，但分隔它们的两个分号必须保留。省略初始化或更新，就不在相应位置执行操作；省略条件，相当于条件始终为真。右圆括号后不加分号，结束花括号后也不需要分号。与 while 一样，花括号及其内容可以整体换成一条语句。

下面的程序用 num 保存当前要计算立方的整数：

```c
#include <stdio.h>

int main(void) {
    int num;
    for (num = 1; num <= 6; num++) {
        printf("%d: %3d\n", num, num * num * num);
    }
    printf("after: %d\n", num);
    return 0;
}
```

```
1:   1
2:   8
3:  27
4:  64
5: 125
6: 216
after: 7
```

第一次执行 for 时，先用 num = 1 设置初值，再判断 num <= 6。条件为真，输出 1 及其立方；随后执行 num++，使 num 变成 2，再回到条件处判断。此后重复“判断、输出、自增”，不会重新执行 num = 1。

输出 6 的立方后，num++ 把 6 变成 7。再次判断 num <= 6 为假，循环结束，后面的 printf 便输出 after: 7。因此，循环体执行 6 次，条件却要判断 7 次：最后一次判断只负责结束循环。图中的 initialization、test、body 和 update 分别表示初始化、条件判断、循环体和更新操作。

```
[initialization]
       |
       v
  +-> <test?> --n--> [after loop]
  |    | y
  |    v
  |  [body]
  |    |
  |    v
  +--[update]
```

图中更新后的箭头回到上方判断，不会回到初始化。可以在初始化位置声明计数器，

如 `for (int num = 1; ...)`。这样声明的 `num` 只在该 `for` 语句及其循环体范围内可用，循环后不能再输出它。若要在循环结束后读取 `num`，须在循环外声明它。

5.3.2 更新不一定是加 1

计数既可以递减，也可以每次增加一个固定步长：

```
for (int secs = 5; secs >= 0; secs--) {
    printf("%d seconds!\n", secs);
}
for (int n = 2; n <= 60; n += 6) {
    printf("%d ", n);
}
```

第一个循环输出 5 到 0，共 6 行，并不表示程序真的每隔一秒执行一次。第二个循环输出 2、8、14、20、26、32、38、44、50、56；下一项 62 不满足条件。

在 ASCII 字符编码环境中，小写字母的编码连续，可以逐个输出字母及其编码：

```
for (int ch = 'a'; ch <= 'z'; ch++) {
    printf("%c: %d\n", ch, ch);
}
```

这里用 `int` 计数，`%c` 将值显示为字符，`%d` 将同一个值显示为整数。C 不保证所有执行字符集的字母编码都连续。

判断可以检查计算结果，更新也可以同时影响其他变量：

```
for (int num = 1; num * num * num < 300; num++) {
    printf("%d ", num);
}
int x, y = 55;
for (x = 1; y <= 75; y = (++x * 5) + 50) {
    printf("(%d,%d) ", x, y);
}
```

第一个循环检查的是 `num` 的立方是否小于 300。`num` 为 1 到 6 时满足条件；增加到 7 后，$7^3 = 343$，条件为假，所以输出的是 1 到 6。

第二个循环用 `x` 表示序号，用 `y` 控制是否继续。开始时 `x = 1`、`y = 55`，先输出 `(1,55)`。更新表达式先执行 `++x`，再按 $y = 5x + 50$ 计算新的 `y`，所以下一轮输出 `(2,60)`。依此类推，输出 `(5,75)` 后，`x` 增至 6，`y` 变成 80，条件 `y <= 75` 为假，循环结束。

5.3.3　省略初始化、条件或更新

for 括号里的初始化、条件和更新都可以省略，但分隔它们的两个分号必须保留。省略某一项后，要看程序在什么位置完成相应的工作。例如，下面省略了括号中的更新操作，在循环体中改变 ans：

```
int ans = 2;
for (int n = 3; ans <= 25; ) {
    ans *= n;
}
printf("%d\n", ans);
```

n 只在初始化时被设为 3，此后没有改变；每一轮改变的是 ans。循环体把 ans 乘以 3，使它依次从 2 变成 6、18、54。下一次判断 ans <= 25 为假，循环结束，输出 54。继续条件是在每轮执行之前检查的，所以最后一次更新后的值可以超过 25。

若省略中间的条件，for 就不再通过这个位置的判断结束循环；语法上把省略的条件视为真。例如 for (;;) 没有初始化和更新操作，也没有显式条件，通常在循环体内用 break 退出循环，或用 return 结束当前函数。

初始化位置不限于给变量赋值，也可以放其他只需执行一次的表达式。下面的 printf 只在进入 for 时输出一次 Start；此后反复读入整数，读到 6 时结束循环，读取失败时由 return 结束程序：

```
int num = 0;
for (printf("Start\n"); num != 6; ) {
    if (scanf("%d", &num) != 1) {
        return 1;
    }
}
```

若不检查 scanf 的返回值，输入字母后 num 不会更新，字母也留在输入流中，后续各轮会反复读取失败。

若用变量保存每轮的增加量，还要检查这个增加量是否可能变成 0。例如，delta 为正整数时，反复执行 delta = delta * 1 / 2; 会使它越来越小；从 5 开始，依次得到 2、1、0。此后执行 n = n + delta 就不再改变 n。如果循环只能靠 n 增大到某个上限来结束，这样的更新就无法使它停止。

5.3.4 用逗号表达式同时更新两项状态

假设邮件重 1 盎司时，邮资为 10 美分；此后重量每增加 1 盎司，邮资增加 4 美分。盎司是重量单位。要打印重量为 1 到 15 盎司的邮资表，可以让重量 ounces 从 1 开始、费用 cost 从 10 开始，每轮分别增加 1 和 4。cost 以美分为单位保存整数费用，输出时除以 100.0，换算成美元。

```
#include <stdio.h>
#define FIRST_OZ 10
#define NEXT_OZ 4

int main(void) {
    int ounces, cost;
    for (ounces = 1, cost = FIRST_OZ;
         ounces < 16;
         ounces++, cost += NEXT_OZ) {
        printf("%2d oz: $%.2f\n", ounces, cost / 100.0);
    }
    return 0;
}
```

首行是 1 oz: $0.10，末行是 15 oz: $0.66，字段宽度还会在首行数字前补一个空格。初始化中的逗号使两次赋值依次进行，更新中的逗号使重量增加后再增加费用。逗号表达式能安排多个动作，但循环的判断仍只由中间的 ounces < 16 决定。

5.4 do-while：先做一次，再决定是否继续

门禁程序先提示用户输入密码，再判断密码是否正确。密码错误时重新询问，直到输入正确。这种先执行操作、再决定是否重复的过程，可以用 do-while 表达。

do-while 的常用形式是：

```
do {
    /* loop body */
} while (condition);
```

condition 代表条件表达式，注释处填入循环体。程序先执行循环体，再判断条件：非零时返回循环体，0 时结束循环。因此，只要执行到这条循环语句，就会先进入循环体，不能靠末尾条件跳过第一次执行。while (condition) 后面的分号是这一语法的一

部分，不能省略。花括号及其内容可以整体换成一条语句；使用花括号能使循环体与末尾的条件更容易辨认。

下面把循环体写成提示并读取密码的操作：

```
#include <stdio.h>

int main(void) {
    const int secret_code = 13;
    int code_entered;
    do {
        printf("Enter the code:\n");
        if (scanf("%d", &code_entered) != 1) {
            printf("Input ended.\n");
            return 1;
        }
    } while (code_entered != secret_code);
    printf("Accepted.\n");
    return 0;
}
```

输入 15、14、13 时，程序询问三次，第三次后输出 Accepted.。第一次输入失败就执行 return，不会用未赋值的 code_entered 判断。

程序执行到 do 时，先进入花括号中的循环体，执行到末尾才检查 code_entered != secret_code。密码错误，条件为真，回到循环体再次询问；密码正确，条件为假，离开循环。由于入口处没有条件判断，循环体至少会开始执行一次；如果体内执行了 return，则直接结束函数，不再检查末尾条件。

结尾 while (...); 中的分号是 do-while 语法必需的。它与普通 while 条件后误加的分号不同：普通 while 的分号会充当空循环体，而 do-while 的循环体已经写在前面的花括号中。

这三个例子分别体现了三种循环的常见用途：立方表的起点、终点和更新方式明确，适合用 for；求和程序在每次读取成功后才处理数据，适合用 while；询问密码要先做一次，再决定是否重复，适合用 do-while。

三种循环也能完成许多相同的任务。例如，立方表中的 for 可以改成 while：把 num = 1 放在循环之前，用 num <= 6 作条件，把 num++ 放在循环体末尾。这样的改写保留了原来的执行顺序。但若体内有跳过本轮剩余语句的 continue，就要重新检查更新是否还会执行：for 中的 continue 会转到更新位置，而 while 中的 continue 会直接回到条件处。

5.5　嵌套循环：一行怎样变成一张表

打印 6 行字母，每行 10 个，可以用外层循环决定行，内层循环决定该行输出的字符。**嵌套循环**就是循环体中再包含循环。以下采用 ASCII 环境：

```
#include <stdio.h>
#define ROWS 6
#define CHARS 10

int main(void) {
    for (int row = 0; row < ROWS; row++) {
        for (int ch = 'A'; ch < 'A' + CHARS; ch++) {
            putchar(ch);
        }
        putchar('\n');
    }
    return 0;
}
```

输出共 6 行，每行都是 ABCDEFGHIJ。外层 row 为 0 时，内层 ch 从'A' 开始，连续输出 10 个字母；内层循环结束后，才执行 putchar('\n') 换行。随后外层把 row 增加到 1，再进入下一轮，内层又从 ch = 'A' 重新开始。因此，每一行都从 A 开始。

外层执行 6 轮，每轮的内层循环输出 10 个字母，所以 putchar(ch) 共执行 $6 \times 10 = 60$ 次，换行执行 6 次。若把换行放进内层，每输出一个字母就会换行；若把换行移到整个外层循环之后，则要等 60 个字母全部输出完才换行。

现在改为打印 6 行逐渐缩短的字母：第一行从 A 到 F，第二行从 B 到 F，依次类推，最后一行只有 F。每一行的终点都是 F，起点则向后移动一个字母。用从 0 开始计数的 row 表示行的位置，起始字符就可以写成'A' + row：

```c
#include <stdio.h>

int main(void) {
    for (int row = 0; row < 6; row++) {
        for (int ch = 'A' + row; ch < 'A' + 6; ch++) {
            putchar(ch);
        }
        putchar('\n');
    }
    return 0;
}
```

```
ABCDEF
BCDEF
CDEF
DEF
EF
F
```

row 从 0 计数，因此第 row+1 行输出 6−row 个字母。总数是 6+5+4+3+2+1 = 21，不是 6 × 6。只有内层每轮次数固定时，才能直接用两个固定次数相乘。

5.6　if 与 else：根据条件选择处理方式

5.6.1　零度以下的日子有多少

回到开头的温度统计任务。每次成功读入一个温度，都要把总天数加 1；只有这个温度低于 0 摄氏度时，才把低温天数加 1。后一项操作由 `if` 控制。

只在条件满足时执行一组操作，可以写成：

```c
if (condition) {
    /* statements */
}
```

condition 代表条件表达式，注释处填入条件成立时要执行的语句。先计算条件：非零时执行花括号内的语句，0 时跳过它们；然后都继续执行整个 `if` 之后的语句。花括号及其内容可以整体换成一条语句。右圆括号后不加分号，结束花括号后也不需要分号。

条件成立与不成立时分别执行不同的操作，用 if-else：

```
if (condition) {
    /* true branch */
} else {
    /* false branch */
}
```

condition 代表条件表达式，两处注释分别代表真分支和假分支中的语句。条件只在进入这个结构时判断一次：非零时执行真分支，0 时执行假分支，两者只执行其一；执行完选中的分支后，继续执行整个结构之后的语句。else 后不写条件，两组花括号及其内容都可以分别换成一条语句。第一组结束花括号与 else 之间不能插入分号，否则会使 else 失去所对应的 if。

温度统计需要用到这两种形式：循环体内用 if 决定是否增加低温天数；循环结束后用 if-else 决定计算百分比，还是提示没有数据。

temperature 保存本次温度，类型为 double，所以读取时使用%lf。all_days 和 cold_days 分别记录总天数和低温天数，都从 0 开始。循环停止后，程序输出总天数和低温天数所占的百分比；若没有读到温度，则输出没有数据的提示。

```
#include <stdio.h>

int main(void) {
    double temperature;
    int cold_days = 0, all_days = 0;
    while (scanf("%lf", &temperature) == 1) {
        all_days++;
        if (temperature < 0.0) {
            cold_days++;
        }
    }
    if (all_days != 0) {
        printf("%d days, %.1f%% below zero.\n",
               all_days, 100.0 * cold_days / all_days);
    } else {
        printf("No data entered.\n");
    }
    return 0;
}
```

依次输入以下温度，最后用 q 结束：

```
12 5.6 -3.2 0 10 2.3 -6 -9.5 5 3 q
```

程序输出：

```
10 days, 30.0% below zero.
```

这 10 个温度中，-3.2、-6 和 -9.5 低于 0，因此循环结束时 all_days 为 10，cold_days 为 3。0 只计入总天数，不计入低温天数，所以百分比是 $3/10 \times 100\% = 30\%$。

表达式 100.0 * cold_days / all_days 先把低温天数与 double 常量 100.0 相乘，再做浮点除法，得到 30.0。若先计算两个整数的 cold_days / all_days，3 / 10 会得到 0，再乘 100 也无法恢复被截去的小数部分。输出格式中的%% 用于打印一个百分号。

else 指定条件为假时执行的语句。到达 if-else 后，两条分支恰好选一条。如果没有数据，程序进入 else，不会计算分母为 0 的比例。条件 all_days != 0 也可以写成 all_days，但明确写出比较更容易看出为什么能够做除法。

5.6.2　多个独立 if 与一条分支链

几个独立的 if 会依次判断各自的条件。前一个 if 执行了自己的分支，也不会阻止后一个 if 继续判断，所以多个分支可能先后执行。

需要依次检查多个条件时，可以把 if-else 接成以下形式：

```
if (condition1) {
    /* first branch */
} else if (condition2) {
    /* second branch */
} else {
    /* remaining cases */
}
```

condition1 和 condition2 代表依次检查的条件。先判断第一个条件；若为真，执行第一个分支，不再判断后面的条件。只有第一个条件为假时，才判断第二个条件；第二个也为假时，才执行最后的 else 分支。因此，整个结构至多选择一个分支。else if 部分可以继续增加，最后的 else 可以省略；省略后，所有条件都为假时就不执行任何分支。else if 是 else 后接一条新的 if 语句，并不是另一个 C 关键字。

例如，对 95 分分别判断“至少 60 分”和“至少 90 分”，两个条件都为真；若用

两个独立的 if，两处输出都会执行。若每个成绩只能归入一个档次，就应使用分支链。以下假设 score 已经是 0 到 100 之间的整数：

```
int score = 95;
if (score >= 90) {
    printf("90-100\n");
} else if (score >= 60) {
    printf("60-89\n");
} else {
    printf("0-59\n");
}
```

95 分只会进入第一个分支。能到达第二个判断，说明 score >= 90 已经为假，所以第二个分支对应 60 到 89 分。

按“至少多少分”判断时，应先检查较高的门槛。若先判断 score >= 60，95 分就会被第一条分支接收，后面的 score >= 90 没有机会执行。

5.6.3 else 究竟与哪一个 if 配对

C 按语法结构匹配 else，不按缩进匹配。else 与同一结构中最近的、尚未配对的 if 结合。下面片段假设 number 已赋值：

```
if (number > 6)
    if (number < 12)
        printf("You're close!\n");
    else
        printf("Sorry, you lose a turn!\n");
```

number 为 5 时，外层条件就失败，两条输出都不执行；为 10 时输出 You're close!；为 15 时进入外层，再走内层的 else。

如果要让 else 属于外层，就必须用花括号封闭内层语句：

```
if (number > 6) {
    if (number < 12) {
        printf("You're close!\n");
    }
} else {
    printf("Sorry, you lose a turn!\n");
}
```

修改后 5 会输出 `Sorry, you lose a turn!`，10 仍输出 `You're close!`，15 则没有输出。花括号改变的是语句归属，不只是版面。

分支要执行两条语句时，必须用花括号把它们组成复合语句。独立的 `x++;` 若放在 `if` 分支与 `else` 之间，会使 `else` 无法匹配。应把输出和自增语句放在同一对花括号内。

5.6.4　阶梯电费为什么只对超出的部分加价

假设电价如下：前 360 千瓦时每千瓦时 0.12589 美元；超过 360 而不超过 680 的部分，单价 0.17901 美元；超过 680 的部分，单价 0.20971 美元。千瓦时是电能单位。

记用电量为 k，费用为 B，则

$$B=\begin{cases}0.12589k, & 0\le k\le 360,\\ 0.12589\times 360+0.17901(k-360), & 360<k\le 680,\\ 0.12589\times 360+0.17901\times 320+0.20971(k-680), & k>680.\end{cases}$$

费用按各段用电量分别计算。例如用电 580 千瓦时，其中 360 千瓦时按第一档单价收费，剩余 $580-360=220$ 千瓦时按第二档单价收费，不能把 580 全部乘以第二档单价。超过 680 千瓦时时，前两段分别是 360 千瓦时和 $680-360=320$ 千瓦时，只有剩余部分使用第三档单价。

程序中的 `kwh` 保存用电量，`bill` 保存费用；`BASE1` 是前 360 千瓦时的费用，`BASE2` 是前 680 千瓦时的累计费用。负的用电量没有意义，应先报告错误并结束程序。

```c
#include <stdio.h>
#define RATE1 0.12589
#define RATE2 0.17901
#define RATE3 0.20971
#define BREAK1 360.0
#define BREAK2 680.0
#define BASE1 (RATE1 * BREAK1)
#define BASE2 (BASE1 + RATE2 * (BREAK2 - BREAK1))

int main(void) {
    double kwh, bill;
    if (scanf("%lf", &kwh) != 1) {
        return 1;
    }
    if (kwh < 0.0) {
        printf("Invalid usage.\n");
        return 1;
    }
    if (kwh <= BREAK1) {
        bill = RATE1 * kwh;
    } else if (kwh <= BREAK2) {
        bill = BASE1 + RATE2 * (kwh - BREAK1);
    } else {
        bill = BASE2 + RATE3 * (kwh - BREAK2);
    }
    printf("The charge for %.1f kWh is $%.2f.\n", kwh, bill);
    return 0;
}
```

580 千瓦时的费用是 $45.3204 + 0.17901 \times 220 = 84.7026$，保留两位小数输出 84.70。360 属于第一档，680 属于第二档。条件中的 `<=` 把这两个边界值分别包含在前一档内。

5.7 把多个条件组合起来

5.7.1 逻辑与、逻辑或、逻辑非

筛选 90 到 100 分的成绩时，成绩既要不低于 90 分，又要不高于 100 分。这两个条件必须同时成立，因此写成 `score >= 90 && score <= 100`。`&&` 称为**逻辑与**，只有两边都为真，整个表达式才为真。

筛选无效成绩时，低于 0 分和高于 100 分只要满足一项，就应判为无效，因此写成 `score < 0 || score > 100`。`||` 称为**逻辑或**，至少一边为真，整个表达式就为真。

! 称为**逻辑非**，对一个条件取反：原条件为真，结果为假；原条件为假，结果为真。例如，!(score < 0) 表示成绩不小于 0。逻辑运算把 0 视为假、非零值视为真；运算结果的类型为 int，结果为假时取 0，为真时取 1。下表中的 a、b 分别代表两个条件。

a 的真假	b 的真假	a && b	a || b	!a
假	假	0	0	1
假	真	0	1	1
真	假	0	1	0
真	真	1	1	0

例如，5 > 2 为真，4 > 7 为假。因此，用 && 连接时结果为 0，用 || 连接时结果为 1。!(4 > 7) 把假变成真，结果也为 1。直接写关系比较 4 <= 7，可以表达同样的判断。

不能写成 90 <= score <= 100。它按 (90 <= score) <= 100 组合，左边第一次比较只能得到 0 或 1，两者都小于等于 100，于是整个条件总为真。

5.7.2　优先级规定分组，短路规定是否继续求值

一个表达式中出现多个运算符时，优先级规定它们怎样组合。这里用到的优先级从高到低依次为：逻辑非!，乘除和取余，加减，关系比较（<、<=、>、>=），相等比较（==、!=），逻辑与 &&，逻辑或 ||，条件运算符?:，赋值，逗号运算符。其中，?: 的用法将在下一节介绍。! 只作用于一个操作数，属于一元运算符；它与前缀自增的优先级相同，低于后缀自增。例如

```
a > b || b > c && b > d
```

按 (a > b) || ((b > c) && (b > d)) 组合。优先级容易误读时可以明确加括号。

&& 和 || 还规定了哪些部分需要计算，这称为**短路求值**。两者都先计算左边：对于 &&，左边为假时，整个条件已经不可能为真，因此不再计算右边；对于 ||，左边为真时，整个条件已经成立，也不再计算右边。只有左边还不能决定最终真假时，才继续计算右边。此时，左边的计算及其中的赋值、自增等变量改变，都已经完成。

短路求值可以让前一个条件保护后面的运算。例如，设 number 是已经赋值的 int 变量：

```
if (number != 0 && 12 / number == 2) {
    printf("Matched.\n");
}
```

当 number 为 0 时，左边的比较为假，右边的除法不会执行；只有 number 非零时，程序才计算 12 / number。因此，这个条件避免了除以 0。若把两项颠倒，程序就会先

做除法，后面的非零检查已经来不及提供保护。这里采用整数除法，例如 number 为 5 时，12 / 5 也得到 2，所以条件成立并不意味着 number 一定等于 6。

再看 x++ < 10 && x + y < 20。设 x、y 都已赋值，且加法和自增不会溢出。左边用 x 的原值与 10 比较，并在开始计算右边之前完成自增；若比较为真，右边使用增加后的 x。例如，原来 x = 9，y = 10 时，左边的 9 < 10 为真，随后右边计算 10 + 10 < 20，结果为假。若原来 x = 10，左边已经为假，右边不再计算，但 x 仍会增加到 11。短路跳过的是右边，不会撤销左边已经发生的自增。

这里的先后保证只针对 &&、|| 连接的左右两部分，不能据此推断所有运算都从左到右执行。例如，普通加法两边的计算和函数各个实参的计算，并不因此获得同样的顺序保证。还要注意，单个 & 和 | 对整数的各个二进制位进行运算，没有上述短路规则，不能用来替代 && 和 ||。

5.7.3 读字符时，先存入，再判断

有时需要先读过输入开头的空格和水平制表符，再处理后面的内容。getchar() 每次读取一个字符；读取成功时返回字符值，遇到文件结束或读取错误时返回 EOF。因此，用 int 变量 c 接收返回值，既能保存字符值，也能与 EOF 比较。下面的循环只跳过空格和水平制表符：

```
int c;
while ((c = getchar()) == ' ' || c == '\t')
    ;
```

每次判断都先调用 getchar()，把返回值保存到 c，再检查是否为空格。如果是空格，逻辑或已经为真，直接进入空循环体，然后读取下一个字符；如果不是空格，才继续检查是否为水平制表符。

若循环因读到其他字符而结束，这个字符**已经被读取并保存在 c 中**，后续处理应先使用 c，否则就会漏掉它。若循环因输入结束而退出，c 保存的则是 EOF。换行既不是空格，也不是水平制表符，因此会使循环停止；这个循环并没有跳过所有种类的空白字符。

接下来对一行文本做一个简单变换：空格原样输出，其他字符则把编码加 1 后输出。下面假定使用 ASCII 编码，输入一行可打印字符，并以换行结束；若先遇到文件结束，也停止读取。这里改变的是字符编码，并不保证变换后的结果仍是字母或可打印字符。

```c
#include <stdio.h>

int main(void) {
    int ch;
    while ((ch = getchar()) != EOF && ch != '\n') {
        if (ch == ' ') {
            putchar(ch);
        } else {
            putchar(ch + 1);
        }
    }
    putchar('\n');
    return 0;
}
```

输入 `Hello, the world!`，输出 `Ifmmp- uif xpsme"`。字符按编码加 1，所以 `z` 会变成 `{`，不会回到 `a`。`putchar` 接收 `int`，按转换为 `unsigned char` 后的字符值输出。

这类条件中，赋值表达式周围的括号不能省略。以只检查换行为例，`ch = getchar() != '\n'` 会先比较读到的值是否为换行，再把比较结果 0 或 1 存入 `ch`，原字符值没有保存下来。写成 `(ch = getchar()) != '\n'`，才会先把读取结果存入 `ch`，再检查是否为换行。完整程序还必须像上例那样检查 `EOF`，以便在输入结束时退出。

5.7.4　句号前，除了引号还剩多少字符

现在统计 ASCII 文本中第一个句号之前的字符数，但不计入单引号和双引号。一个字符要被计数，必须同时满足“不是单引号”和“不是双引号”，因此两个比较之间使用 `&&`。程序遇到句号或文件结束时停止；空格和换行都可以被计数。

```c
#include <stdio.h>

int main(void) {
    int ch;
    unsigned long count = 0;
    while ((ch = getchar()) != EOF && ch != '.') {
        if (ch != '\'' && ch != '"') {
            count++;
        }
    }
    printf("There are %lu characters.\n", count);
    return 0;
}
```

字符常量'\'' 表示一个单引号，其中的反斜杠用来避免把这个单引号误认作字符常量的结束符。'"' 表示一个双引号。

输入"I don't know." He said. 时，程序在第一个句号处停止。句号之前有 9 个字母、2 个空格、一个单引号和一个双引号；排除两种引号后，计数为 11。句号本身不计数，句号后面的文本也不再读取。

如果把 && 改成 ||，单引号虽然不满足左边，却满足“不是双引号”的右边，仍会被计数；双引号同样会通过左边的比较。其他字符则同时通过两个比较。这样就没有排除任何字符。

本例限定 ASCII 文本，是因为 getchar 逐字节读取输入。UTF-8 中的一个汉字通常占多个字节，直接这样累加得到的不是汉字个数。

5.8 条件运算符：从两个值中选一个

条件表达式有三个操作数，写成：

```c
condition ? value1 : value2
```

先计算 condition：若它非零，只计算 value1；若它为 0，只计算 value2。表达式的值由选中的一项给出，另一项不会被计算。例如，设 a、b 和 max 都是 int，max = a > b ? a : b; 的含义是：若 a > b，把 a 赋给 max；否则，把 b 赋给 max。这种写法适合从两个值中作简短选择。

再设 x、y 都是 int。x = y < 0 ? y : -y; 在 y 为负时保留原值，在 y 非负时

取相反数，所以 `x` 得到 y 的负绝对值。例如，$y=-3$ 和 $y=3$ 都得到 -3，$y=0$ 得到 0。

两个候选值不一定具有相同类型。当两项都是算术类型时，条件表达式还要根据两项的类型确定共同的结果类型。例如 `1 ? 2 : 3.5` 只计算并选中 2，但整个表达式的类型为 `double`，值为 `2.0`。`3.5` 所在的一项虽然不被计算，它的类型仍参与决定结果类型。

假设一桶油漆可覆盖 200 平方英尺，平方英尺是面积单位。需要涂刷的面积为非负整数，不满一桶的部分也必须购买一整桶。先求整数商，再根据余数决定是否增加 1：

```
#include <stdio.h>
#define COVERAGE 200

int main(void) {
    int area;
    while (scanf("%d", &area) == 1) {
        if (area < 0) {
            printf("Invalid area.\n");
        } else {
            int cans = area / COVERAGE;
            cans += (area % COVERAGE == 0) ? 0 : 1;
            printf("You need %d %s of paint.\n", cans,
                   cans == 1 ? "can" : "cans");
        }
    }
    return 0;
}
```

例如，面积为 215 平方英尺时，整数除法 `215 / 200` 得到 1，表示一桶能覆盖的完整部分；余数 15 表示还有面积未被覆盖，因此再加 1，得到 2 桶。面积为 200 时余数为 0，只需 1 桶；面积为 0 时商和余数均为 0，需要 0 桶。

输出语句中的 `cans == 1 ? "can" : "cans"` 是另一个条件表达式：一桶使用英语单数 `can`，其他桶数使用复数 `cans`，选中的字符串由`%s` 输出。计算桶数时先求商和余数，还避免了 `(area + 199) / 200` 在 `area` 接近整数上限时可能发生的加法溢出。

5.9　把循环和分支合起来：约数与素数

给定一个整数，先判断它是否为素数；对于大于 1 的非素数，还要列出它的所有非平凡正约数。正整数 d 能整除整数 n，就称 d 为 n 的一个正约数。**素数**是大于 1、正约数只有 1 和自身的整数；0、1 和负整数都不是素数。

对于大于 1 的 n，1 和 n 本身一定是约数。把它们排除后，剩下的约数称为**非平凡正约数**，范围是 2 到 $n-1$。只要找到一个这样的约数，就能确定 n 不是素数；如果不存在，n 就是素数。

最直接的方法是让候选约数 `divisor` 从 2 开始，依次尝试到 $n-1$。当 `n % divisor == 0` 时，说明余数为 0，找到了一个约数。

不过，找到一个约数时，其实可以同时找到另一个。例如，36 能被 2 整除，就得到约数对 2 和 18；能被 3 整除，就得到 3 和 12。一般地，若 $n = dq$，找到 d 也就找到了 $q = n/d$。如果 d 和 q 都大于 $\sqrt{n}$，它们的乘积就会大于 n，与 $dq = n$ 矛盾。因此，每对约数至少有一个不超过 $\sqrt{n}$。只需尝试 2 到 $\sqrt{n}$ 之间的整数，再由除法求出配对约数，就不会漏掉非平凡正约数。

在程序中，用 `num` 保存待检查的整数。候选约数不超过平方根，可以写成 `divisor * divisor <= num`；但计算平方可能超出 `int` 的范围，因此改写为 `divisor <= num / divisor`。这里 `num` 和 `divisor` 都是正整数，即使整数除法舍去小数部分，两种写法判断的整数范围也相同。

程序还需要记住是否发现过约数。`is_prime` 就是记录这个状态的**标志变量**：每开始检查一个大于 1 的新数，就把它设为 1，表示尚未发现非平凡正约数；找到约数后改为 0，并保持到当前整数的检查结束。只有完整查完候选范围后仍为 1，才能报告素数。

```c
#include <stdio.h>

int main(void) {
    int num;
    while (scanf("%d", &num) == 1) {
        if (num < 2) {
            printf("%d is not prime.\n", num);
        } else {
            int is_prime = 1;
            for (int divisor = 2;
                 divisor <= num / divisor; divisor++) {
                if (num % divisor == 0) {
                    int partner = num / divisor;
                    printf("%d", divisor);
                    if (partner != divisor) {
                        printf(" %d", partner);
                    }
                    putchar('\n');
                    is_prime = 0;
                }
            }
            if (is_prime) {
                printf("%d is prime.\n", num);
            }
        }
    }
    return 0;
}
```

输入 36，输出约数对 `2 18`、`3 12`、`4 9`，然后单独输出 6。$6 \times 6 = 36$，一对相同约数只输出一次。这些输出覆盖全部非平凡正约数，但不是从小到大的全局排序。输入 149 会报告素数；输入 30077 会输出 `19 1583`。若只需要判断是否为素数，找到第一个约数就可以结束循环；若要求列出全部约数，则必须继续查找。

5.10　continue：这条记录不处理，接着读下一条

5.10.1　成绩不合范围时，跳过统计

录入成绩时，`scanf` 可以成功读入 118，但 118 超出了 0 到 100 分的有效范围。程序应报告这条成绩无效，并直接接收下一条，不能把 118 计入总分、人数或最高分。

continue 语句的语法是：

```
continue;
```

分号不能省略，continue 后面不带条件或数值。它只能出现在循环体中，执行后跳过最近一层循环在本轮中尚未执行的语句。接下来执行的位置取决于循环形式：while 转到条件判断，do-while 转到末尾的条件判断，for 先执行更新表达式，再判断条件。嵌套循环中的 continue 只影响最内层的所在循环；是否进入下一轮，仍由相应的条件判断决定。

在下面的成绩统计程序中，continue 使程序回到 while 条件，再次调用 scanf 读取成绩。

total 保存总分，count 保存有效成绩数；min 和 max 分别保存目前遇到的最低分和最高分。

```
#include <stdio.h>

int main(void) {
    double score, total = 0.0;
    double min = 100.0, max = 0.0;
    int count = 0;
    while (scanf("%lf", &score) == 1) {
        if (score < 0.0 || score > 100.0) {
            printf("Invalid: %.1f\n", score);
            continue;
        }
        min = score < min ? score : min;
        max = score > max ? score : max;
        total += score;
        count++;
    }
    if (count > 0) {
        printf("count=%d, average=%.1f\n", count, total / count);
        printf("min=%.1f, max=%.1f\n", min, max);
    } else {
        printf("No valid scores.\n");
    }
    return 0;
}
```

输入 `12 35 118 75 q`，118 被拒绝，三次有效输入的总和为 122，平均分保留一位小数为 40.7，最低分 12.0，最高分 75.0。`continue` 跳过了本轮的极值更新、总分累加和计数，然后回到 `while` 条件，重新执行 `scanf`。

为什么 `min` 的初值是 100，而 `max` 的初值是 0？有效成绩都在 0 到 100 之间，所以第一条有效成绩与这两个初值比较后，`min` 和 `max` 都会等于这条成绩；后续记录再逐步更新最低分和最高分。若数据没有已知的取值范围，就应使用第一条有效记录初始化这两个变量。

若一条有效成绩也没有读到，100 和 0 只是初值，不能作为统计结果输出。因此，程序先检查 `count > 0`，既避免用 0 作除数求平均分，也避免报告没有实际意义的最高分和最低分。

这段程序也能不使用 `continue`：在无效分支后加 `else`，把四条统计语句放进去；或者只在成绩位于有效区间时执行统计。

5.10.2　continue 之后，for 与 while 去往不同位置

所在循环	`continue` 后执行	被跳过的部分
`while`	重新求入口条件	循环体剩余语句
`do-while`	求末尾条件	循环体剩余语句
`for`	先更新，再求条件	循环体剩余语句

下面比较两个看起来很相似的字符输出程序。第一个程序忽略换行，最多输出 10 个其他字符；若不足 10 个就遇到文件结束，也会停止。`count` 只在实际输出一个字符后增加：

```
#include <stdio.h>

int main(void) {
    int ch, count = 0;
    while (count < 10 && (ch = getchar()) != EOF) {
        if (ch == '\n') {
            continue;
        }
        putchar(ch);
        count++;
    }
    return 0;
}
```

读到换行时，continue 使程序直接返回 while 条件，跳过 putchar(ch) 和 count++。因此，换行既不输出，也不占用 10 个字符的输出限额。

第二个程序把 count++ 移到了 for 括号中的更新位置。即使循环体执行了 continue，程序仍会先执行这个更新，再判断条件。因此，每成功读取一个字符，计数都会增加，换行也计入其中；此时 count 记录的是**成功读取字符数**：

```
#include <stdio.h>

int main(void) {
    int ch;
    for (int count = 0;
         count < 10 && (ch = getchar()) != EOF; count++) {
        if (ch == '\n') {
            continue;
        }
        putchar(ch);
    }
    return 0;
}
```

给两个程序分别输入字母 ab，再输入换行和字母 cdefghijk。第一个程序输出 abcdefghij，共 10 个字母；第二个程序的前 10 次成功读取中包含一个换行，因此只输出 abcdefghi 这 9 个字母。把 for 机械改写成循环末尾自增的 while，就可能改变这种行为。

若只需要丢弃本行剩余字符，可以写下面的空循环。换行和文件结束都应列入停止条件，否则最后一行没有换行时可能反复读取 EOF：

```
int ch;
while ((ch = getchar()) != '\n' && ch != EOF)
    ;
```

用 continue; 作为这个循环的唯一循环体也合法，不过这里的空语句已经足以表达“动作全在条件里”。在嵌套循环中，continue 只影响最近一层循环，不会自动开始下一轮外层循环。

5.11　break：结束当前循环

5.11.1　长读到了，宽却没有读到

连续计算矩形面积时，每次必须成功获得长和宽。如果长度已读入，宽度输入却失败，就应结束这次连续计算。

break 语句的语法是：

```
break;
```

分号不能省略，break 后面不带条件或数值。它只能出现在循环体或 switch 语句内部，执行后立即结束从当前位置向外遇到的最近一层循环或 switch，转去执行该结构之后的语句。若只想在某个条件成立时退出，应把 break; 放在 if 分支中；if 本身不改变 break 的作用范围。

长和宽的单位为米，取值均须大于 0 且不超过 10000。

```
#include <stdio.h>

int main(void) {
    double length, width;
    while (scanf("%lf", &length) == 1) {
        if (scanf("%lf", &width) != 1) {
            printf("Missing width.\n");
            break;
        }
        if (length <= 0.0 || length > 10000.0 ||
            width <= 0.0 || width > 10000.0) {
            printf("Invalid dimensions.\n");
            continue;
        }
        printf("Area = %.2f square metres.\n", length * width);
    }
    printf("Done.\n");
    return 0;
}
```

输入 1.2 3 1 q 时，程序先读到第一组长 1.2、宽 3，输出面积 3.60。第二组的长 1 已经读入，但读取宽时遇到 q，转换失败，width 没有得到新的值。程序输出 Missing

width.，执行 break 离开循环，所以不会拿上一次的宽 3 来计算第二个矩形的面积。q 仍留在输入流中，但这个程序已不再读取它。

离开循环后，程序继续执行后面的输出语句，因此仍会打印 Done.。break 只结束循环；return 则离开当前函数，从 main 返回时会结束程序的正常执行。

5.11.2 跳出一层，不等于跳出全部层次

break 也跳过循环体的剩余部分，但不会像 continue 那样进入下一轮。在 for 中，break 连括号中的更新都跳过。以下片段结束后，i 为 3：

```
int i;
for (i = 0; i < 10; i++) {
    if (i == 3) {
        break;
    }
}
```

如果把 break 换成 continue，则只有 i == 3 那轮的剩余循环体被跳过，更新仍执行，最后 i 会变成 10。

在多重循环中，内层找到结果后，外层循环不会因此自动结束。可以用一个标志变量让外层知道已经找到结果。下面让 a 和 b 分别取 1 到 6，按 a 从小到大、同一个 a 下 b 从小到大的顺序，寻找第一组乘积为 12 的数。found 初值为 0，找到后改为 1；外层条件中的!found 要求继续搜索时尚未找到结果。

```
int found = 0;
for (int a = 1; a <= 6 && !found; a++) {
    for (int b = 1; b <= 6; b++) {
        if (a * b == 12) {
            printf("%d %d\n", a, b);
            found = 1;
            break;
        }
    }
}
```

程序先尝试 a = 1 下的六个 b，都不成功；到 a = 2, b = 6 时，输出 2 6，把 found 改为 1，再用 break 离开内层的 b 循环。外层仍会执行自己的 a++；再次判断条件时，!found 为假，外层才结束。

另一种写法是在内层循环之后检查 found，若已经找到，就再执行一条 break 退

出外层。无论采用哪种写法，结束外层都需要单独安排；一条 break 只影响最近一层循环。

5.12　switch：在若干离散值中选入口

5.12.1　case 标签不是一条条独立的 if

按一个整数表达式的值选择入口，可以使用 switch。每个分支末尾用 break 结束选择，是一种常用写法：

```
switch (expression) {
    case constant1:
        /* first branch */
        break;
    case constant2:
        /* second branch */
        break;
    default:
        /* remaining cases */
        break;
}
```

expression 代表用于选择入口的整数表达式，constant1 和 constant2 代表各入口对应的整数常量表达式；这些英文名称都是占位符。case 值和 default 之后写冒号，不写分号。default 最多出现一次，也可以省略。

执行时先计算 expression 的值，再转到匹配的 case 标签之后，从那里向下执行。若没有匹配项，就从 default 之后开始；若也没有 default，直接执行 switch 之后的语句。这里的 break; 使程序离开 switch。break 不是 case 语法强制要求的部分；省略后，程序会继续向下执行，遇到下一个 case 或 default 标签也不会自动停止。

switch 根据一个整数值选择入口，因此控制表达式必须具有整数类型，例如 int 或 char，不能直接使用浮点数或字符串。控制表达式还会进行**整数提升**：char、short 等较小整数类型会转换为能够保存其全部取值的 int，若 int 不能保存全部取值则转为 unsigned int。

case 后面的值必须由**整数常量表达式**给出。它由 C 规定允许使用的常量和运算构成，值能在编译时确定。这里可以使用整数字面量、字符常量及它们的整数算术组合，例如 case 'a': 和 case 7 + 5:。同一个 switch 中的标签值在转换到提升后的控制表达式类型后不能重复，否则同一个值就对应了两个入口。普通变量、浮点常量和字符

串不能直接作为标签值。在 C11 中，const int 定义的是只读变量，也不能仅凭 const 就把它当作 case 所需的整数常量表达式。

下面做一个字母提示程序：每行只用第一个字符作选择，处理后读完并丢弃这一行余下的字符。输入小写字母 a、b、c、d 时，分别输出对应单词；输入其他小写字母时，提示尚未收录；输入非小写字母时，提示只能使用小写字母。空行直接跳过，行首为 # 或输入结束时停止。字母范围判断采用 ASCII 编码环境。

```
#include <stdio.h>

int main(void) {
    int ch, rest;
    while ((ch = getchar()) != EOF && ch != '#') {
        if (ch == '\n') {
            continue;
        }
        if (ch >= 'a' && ch <= 'z') {
            switch (ch) {
                case 'a': printf("a for apple\n"); break;
                case 'b': printf("b for boy\n"); break;
                case 'c': printf("c for cat\n"); break;
                case 'd': printf("d for dog\n"); break;
                default:  printf("That's a stumper!\n"); break;
            }
        } else {
            printf("Lowercase letters only.\n");
        }
        while ((rest = getchar()) != '\n' && rest != EOF)
            ;
    }
    printf("Bye!\n");
    return 0;
}
```

依次输入 b、d、x、H、#，每项各占一行，分别输出 b for boy、d for dog、未收录提示、大写输入提示和 Bye!。这里 switch 里的 break 离开的是 switch，之后还会执行丢弃行尾的循环，再读下一行。

5.12.2　没有 break 就继续向下执行

switch 只在开始时根据控制表达式的值选择一次入口。进入某个 case 后，语句就按顺序执行；遇到下一个 case 标签时，不会重新比较，也不会自动停止。如果没有 break 等跳转语句，执行就会继续经过后面的标签，这称为**贯穿执行**。default 同样只是一个标签，前面的语句也可以一直执行到它后面的语句。

有意让几个标签共用同一段代码，可以避免重复。例如统计五种元音字母时，大写和小写可以共享计数语句。读到 # 或文件结束时停止：

```
#include <stdio.h>

int main(void) {
    int ch;
    unsigned long a = 0, e = 0, i = 0, o = 0, u = 0;
    while ((ch = getchar()) != EOF && ch != '#') {
        switch (ch) {
            case 'a': case 'A': a++; break;
            case 'e': case 'E': e++; break;
            case 'i': case 'I': i++; break;
            case 'o': case 'O': o++; break;
            case 'u': case 'U': u++; break;
            default: break;
        }
    }
    printf("A E I O U\n%lu %lu %lu %lu %lu\n", a, e, i, o, u);
    return 0;
}
```

例如，读到小写 a 时，从 case 'a': 进入，继续经过紧邻的 case 'A':，然后执行 a++；读到大写 A 时，则直接进入后一标签，执行同一条计数语句。因此，两种大小写共用一个计数器。

输入 I see under the overseer.#，输出的五个计数依次为 0 7 1 1 1。若只删除 a++ 后面的 break，读到 a 或 A 后，除了增加 a，还会继续执行 e++，直到遇到它后面的 break 才离开。一次输入被错计入两类，统计结果就错了。

如果要判断一个整数是否等于若干指定值，例如字符是否为'a'、'b' 或'c'，可以使用 switch。如果要判断一个区间、比较浮点数，或者组合多个变量的条件，则使用 if-else 更直接。两种写法的运行速度还取决于编译器、优化方式和输入数据，不能仅凭语法形式判断谁更快。

在 switch 嵌套于循环时，还要区分两种跳转：break 作用于最近的循环或 switch；continue 只寻找最近的循环，switch 本身不是循环。

5.13 综合例题：先确定状态，再安排控制流

5.13.1 例题一：三个整数中的最大数

读入三个整数 a、b、c，输出其中的最大值。用 max 保存已经比较过的数中的最大值：先令 max = a；若 b 更大，就用 b 更新它；再以同样的方法检查 c。无论 b 是否大于 a，都必须继续检查 c，因此这里需要两个独立的 if。

```
#include <stdio.h>

int main(void) {
    int a, b, c, max;
    if (scanf("%d%d%d", &a, &b, &c) != 3) {
        return 1;
    }
    max = a;
    if (b > max) {
        max = b;
    }
    if (c > max) {
        max = c;
    }
    printf("max = %d\n", max);
    return 0;
}
```

输入 2 5 9 时，max 先取 2，再更新为 5，最后更新为 9。若把第二个 if 改成 else if，程序在把 max 更新为 5 后就会跳过对 9 的比较，输出错误结果 5。输入-8 -3 -5 时，正确答案是 -3。若把 max 初始化为 0，三个负数都不能使它更新，程序就会错误地输出 0。

5.13.2 例题二：三个整数从大到小排列

读入三个整数，调整 x、y、z 中的值，使输出满足 $x \geq y \geq z$。调整时需要交换两个变量的值。例如先执行 temp = x; 保存 x 的旧值，再执行 x = y;，最后执行 y =

`temp;`，就完成了交换。若直接写 `x = y; y = x;`，第一条赋值已经覆盖 `x` 的旧值，第二条赋值只能把原来的 `y` 再写回 `y`。

```
#include <stdio.h>

int main(void) {
    int x, y, z, temp;
    if (scanf("%d%d%d", &x, &y, &z) != 3) {
        return 1;
    }
    if (x < y) {
        temp = x; x = y; y = temp;
    }
    if (x < z) {
        temp = x; x = z; z = temp;
    }
    if (y < z) {
        temp = y; y = z; z = temp;
    }
    printf("%d %d %d\n", x, y, z);
    return 0;
}
```

第一次比较后，`x` 保存原来 `x` 和 `y` 中较大的数。第二次将 `x` 与 `z` 比较；如果需要交换，换入 `x` 的是一个更大的数，因此这一步结束时 `x` 一定是三数中的最大值。第三次只比较剩下的 `y` 和 `z`，使它们满足 $y \geq z$。输入 `2 9 5` 时，三个变量依次为 $(9,2,5)$、$(9,2,5)$、$(9,5,2)$：第二次比较不满足交换条件，所以数值不变。两个数相等时无需交换，结果仍满足 $x \geq y \geq z$。

5.13.3　例题三：二次方程求根与退化情况

求 $ax^2+bx+c=0$ 的根，先判断 a 是否为 0。如果 $a=0$，方程退化为 $bx+c=0$；若 $b \neq 0$，根为 $-c/b$；若 $a=b=0$，则 $c=0$ 时任意 x 都满足方程，$c \neq 0$ 时无解。

只有 $a \neq 0$ 时，才能使用二次方程的求根公式。先计算**判别式**$\Delta = b^2-4ac$：$\Delta > 0$ 时，两个不同实根为 $(-b+\sqrt{\Delta})/(2a)$ 和 $(-b-\sqrt{\Delta})/(2a)$；$\Delta = 0$ 时，两式得到同一个值 $-b/(2a)$，称为二重实根。

当 $\Delta < 0$ 时，方程没有实根，但有一对复根。复数写成 $u + v\mathrm{i}$，其中 u 为实部，v 为虚部，$\mathrm{i}^2 = -1$。本题的两个复根可写成

$$-\frac{b}{2a} + \frac{\sqrt{-\Delta}}{|2a|}\mathrm{i}, \qquad -\frac{b}{2a} - \frac{\sqrt{-\Delta}}{|2a|}\mathrm{i}.$$

它们实部相同、虚部互为相反数，称为共轭复根。程序用 `re` 保存共同的实部，用非负的 `im` 保存虚部的绝对值，分别打印加号和减号即可。这里不需要把整个复数存入一个变量。

`sqrt` 求平方根，`fabs` 求浮点绝对值，两者由 `<math.h>` 声明。

```
#include <stdio.h>
#include <math.h>

int main(void) {
    double a, b, c;
    if (scanf("%lf%lf%lf", &a, &b, &c) != 3) {
        return 1;
    }
    if (a == 0.0) {
        if (b != 0.0) {
            printf("Linear root: %.6f\n", -c / b);
        } else if (c == 0.0) {
            printf("Infinitely many roots.\n");
        } else {
            printf("No roots.\n");
        }
    } else {
        double delta = b * b - 4.0 * a * c;
        if (delta > 0.0) {
            double root1 = (-b + sqrt(delta)) / (2.0 * a);
            double root2 = (-b - sqrt(delta)) / (2.0 * a);
            printf("Real roots: %.6f %.6f\n", root1, root2);
        } else if (delta == 0.0) {
            printf("Double root: %.6f\n", -b / (2.0 * a));
        } else {
            double re = -b / (2.0 * a);
            double im = sqrt(-delta) / fabs(2.0 * a);
            printf("Complex roots: %.6f + %.6fi, "
                   "%.6f - %.6fi\n", re, im, re, im);
        }
    }
    return 0;
}
```

输入 `1 -3 2` 得到 2 和 1；输入 `1 2 1` 得到重根 -1；输入 `1 -2 5` 得到 $1 \pm 2\mathrm{i}$；输入 `0 2 -6` 得到一次方程的根 3。输入 `0 0 0` 有无穷多解，而 `0 0 1` 无解。

`a == 0.0` 检查的是变量 `a` 中保存的值是否恰好为 0。`delta` 则还经过了乘法和减法，计算中可能产生舍入误差。当数学上的判别式非常接近 0 时，计算所得 `delta` 的正负或是否为 0 可能与精确结果不同，使程序选中不同分支。求根公式中两个相近数相减，也可能使结果丢失有效数字。实际数值计算还要根据系数的大小、输入误差和精度要求，确定多接近 0 才视为 0，并选择能减小计算误差的求根方法。

5.13.4　例题四：一行字符分成三类

读入一行文字，把每次读到的字符分为三类：数字字符`'0'` 到`'9'`；空格或水平制表符`'\t'`；其余字符。分别用 `digits`、`white` 和 `other` 记录三类的数量。遇到换行或文件结束时停止，换行本身不计数。`switch` 让 10 个数字标签共用一次 `digits++`，让空格和制表符标签共用一次 `white++`。

```
#include <stdio.h>

int main(void) {
    int ch;
    unsigned long digits = 0, white = 0, other = 0;
    while ((ch = getchar()) != EOF && ch != '\n') {
        switch (ch) {
            case '0': case '1': case '2': case '3': case '4':
            case '5': case '6': case '7': case '8': case '9':
                digits++; break;
            case ' ': case '\t':
                white++; break;
            default:
                other++; break;
        }
    }
    printf("digits=%lu, white=%lu, other=%lu\n",
           digits, white, other);
    return 0;
}
```

输入 `A1 2`，接着输入一个制表符、`!` 和换行，程序得到 `digits=2, white=2, other=2`。数字是 1 和 2，空白是一个空格和一个制表符，其余字符是 A 和感叹号。每

次成功读取且不是换行的字节，恰好使一个计数器增加 1，因此三个计数器的和等于本行处理的字节数。这里仍是逐字节统计；例如 UTF-8 编码的一个汉字通常会使 `other` 增加多次。

5.13.5　例题五：用牛顿法逐次逼近方程的根

求 $f(x) = 3x^3 + 4x^2 - 2x + 5 = 0$ 的一个实根。**牛顿法**在当前估计值 x_k 附近用切线近似曲线。下标 k 表示已经迭代的次数，$f'(x_k)$ 是该处导数，即切线斜率。切线方程为

$$y = f(x_k) + f'(x_k)(x - x_k).$$

令切线与横轴相交，即令 $y = 0$，解出下一次估计值：

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}, \qquad f'(x) = 9x^2 + 8x - 2.$$

`step` 保存本次修正量 $f(x_k)/f'(x_k)$。从 $x_0 = -2$ 开始，$f(-2) = 1$，$f'(-2) = 18$，第一次更新得到 $-2 - 1/18 \approx -2.055556$。

每轮先根据当前的 `x` 计算函数值 `f` 和导数值 `df`，再求修正量 `step = f / df`，用 `x -= step` 更新估计值。因为计算修正量需要除以 `df`，程序先检查导数的绝对值；若小于 10^{-12}，就报告导数过小并停止，避免继续做这次除法。

更新 `x` 后，还要判断是否继续迭代。**残差** `residual` 是把更新后的 `x` 代回方程得到的 $f(x)$。修正量的绝对值小，说明相邻两次估计接近；残差的绝对值小，说明代回方程后的结果接近 0。本例要求这两项都不超过 `EPS`，才满足停止条件。`EPS` 设为 10^{-6}，是修正量和残差的阈值，并不直接保证近似根与真根的误差小于 `EPS`。`MAX_ITER` 设为 100，使程序至多更新 100 次。

```c
#include <stdio.h>
#include <math.h>
#define EPS 1.0e-6
#define MAX_ITER 100

int main(void) {
    double x = -2.0;
    int converged = 0;
    for (int k = 0; k < MAX_ITER; k++) {
        double f = ((3.0 * x + 4.0) * x - 2.0) * x + 5.0;
        double df = (9.0 * x + 8.0) * x - 2.0;
        if (fabs(df) < 1.0e-12) {
            printf("Derivative too small.\n");
            return 1;
        }
        double step = f / df;
        x -= step;
        double residual = ((3.0 * x + 4.0) * x - 2.0) * x + 5.0;
        if (fabs(step) <= EPS && fabs(residual) <= EPS) {
            converged = 1;
            break;
        }
    }
    if (!converged) {
        printf("Iteration limit reached.\n");
        return 1;
    }
    printf("root = %.6f\n", x);
    return 0;
}
```

程序输出 `root = -2.053319`。代码中的 `((3.0 * x + 4.0) * x - 2.0) * x + 5.0` 展开后就是 $3x^3 + 4x^2 - 2x + 5$，逐层乘加可以减少重复乘法。

能走到循环后判断的有两条路径：满足修正量和残差的阈值时执行 `break`，或者执行满 100 次后由循环条件结束。`converged` 初值为 0，只在满足本例的停止条件时改成 1，因此循环后的判断能够区分“已满足停止条件”和“已用完迭代次数”。若导数过小，程序执行的是 `return 1;`，会直接结束 `main`，不经过循环后的判断。牛顿法并不保证从任意初值出发都能收敛。若扩大系数或初值的范围，还需检查计算结果是否超出浮点数能表示的范围。

5.13.6　例题六：余数条件如何减少查找次数

寻找最小正整数 n，满足除以 3 余 2、除以 5 余 3、除以 7 余 4。

第一种办法按 2、3、4 这样的顺序逐个尝试。1 不满足“除以 3 余 2”，因此可以从 2 开始。循环条件中的三个比较检查三个余数要求，`&&` 要求它们同时成立；外面的 `!` 将结果取反，所以只要候选数还不满足全部要求，就执行 `n++`，尝试下一个数：

```
#include <stdio.h>

int main(void) {
    int n = 2;
    while (!(n % 3 == 2 && n % 5 == 3 && n % 7 == 4)) {
        n++;
    }
    printf("solution = %d\n", n);
    return 0;
}
```

程序输出 53。还可以利用余数条件减少尝试次数：先只检查除以 3 余 2 的数，即 2、5、8、11 等；其中第一个除以 5 余 3 的数是 8。接着从 8 开始，每次增加 15。因为 15 同时能被 3 和 5 整除，这样得到的候选数始终满足前两个条件，只需继续检查除以 7 的余数。

```
#include <stdio.h>

int main(void) {
    int n = 2;
    while (n % 5 != 3) {
        n += 3;
    }
    while (n % 7 != 4) {
        n += 15;
    }
    printf("solution = %d\n", n);
    return 0;
}
```

第二个循环依次检查 8、23、38、53，到 53 时除以 7 余 4，循环结束。为什么每次加 15 不会漏掉更小的解？任意两个同时满足前两条要求的数，它们的差既能被 3 整除，

也能被 5 整除，因此必为 15 的倍数。8 又是满足前两条要求的最小正数，所以 8、23、38、53 这条序列已经按顺序包含了所有需要检查的候选数；第一个满足第三条要求的数 53，就是最小正解。

5.13.7　例题七：最大值第一次出现在第几项

先输入整数 n，说明随后有多少个整数，再输入这 n 个数。程序输出最大值及其第一次出现的位置，位置从 1 开始编号。本例规定 $1 \le n \le 10000$；若没有任何数，就不存在可供比较的最大值。用 `max` 保存目前的最大值，`index` 保存它的位置，`value` 保存本次新读入的数。

```c
#include <stdio.h>

int main(void) {
    int n, value, max, index;
    if (scanf("%d", &n) != 1 || n < 1 || n > 10000) {
        return 1;
    }
    if (scanf("%d", &max) != 1) {
        return 1;
    }
    index = 1;
    for (int i = 2; i <= n; i++) {
        if (scanf("%d", &value) != 1) {
            return 1;
        }
        if (value > max) {
            max = value;
            index = i;
        }
    }
    printf("max=%d, index=%d\n", max, index);
    return 0;
}
```

输入 `5 -8 7 7 2 -1` 时，开头的 5 表示数据项数，随后五个数才是待比较的数据。程序先读入第一项 -8，令 `max=-8`、`index=1`，因此循环从第二项开始。读到第二项 7 时，最大值和位置更新为 7 与 2；第三项也是 7，不满足 `value > max`，位置保持不变。

最后输出 max=7, index=2。若将 > 改成 >=，相等时也会更新位置，这个输入就会得到 index=3，保存的是最大值最后出现的位置。

5.13.8　例题八：50 张纸币怎样凑成 116 元

一元、二元、五元纸币共 50 张，总面值 116 元，一元比二元多 2 张。设三类数量分别为 one、two 和 five。已知

$$\texttt{two} = \texttt{one} - 2, \qquad \texttt{five} = 50 - \texttt{one} - \texttt{two} = 52 - 2\texttt{one}.$$

三类纸币的张数都不能为负。two = one - 2 要求 one ≥ 2，而 five $= 52 - 2$one 要求 one ≤ 26。因此只需让 one 依次取 2 到 26，算出对应的 two 和 five，再检查总金额是否为 116 元。张数总和与数量差已经由上面两个等式保证，不必在 if 中重复判断。

```
#include <stdio.h>

int main(void) {
    for (int one = 2; one <= 26; one++) {
        int two = one - 2;
        int five = 50 - one - two;
        if (one + 2 * two + 5 * five == 116) {
            printf("one=%d, two=%d, five=%d\n", one, two, five);
        }
    }
    return 0;
}
```

结果为一元 20 张、二元 18 张、五元 12 张。检查张数 $20 + 18 + 12 = 50$，金额 $20 + 36 + 60 = 116$，数量差 $20 - 18 = 2$。控制候选范围不仅减少循环次数，也排除了纸币数量为负的无意义状态。

5.13.9　例题九：边长不超过 25 的整数直角三角形

找出三条边长都是正整数、且都不超过 25 的直角三角形。用 a, b 表示两条直角边，c 表示斜边，候选边长必须满足 $a^2 + b^2 = c^2$。斜边长于任意一条直角边；交换两条直角边又不会得到一个新的三角形，因此只检查 $1 \leq a \leq b < c \leq 25$ 的组合，就能避免把 $(3, 4, 5)$ 与 $(4, 3, 5)$ 重复输出。

```c
#include <stdio.h>

int main(void) {
    for (int c = 3; c <= 25; c++) {
        for (int b = 1; b < c; b++) {
            for (int a = 1; a <= b; a++) {
                if (a * a + b * b == c * c) {
                    printf("%d %d %d\n", a, b, c);
                }
            }
        }
    }
    return 0;
}
```

```
3 4 5
6 8 10
5 12 13
9 12 15
8 15 17
12 16 20
15 20 25
7 24 25
```

题目要求全部整数边长三角形，因而保留 $(6,8,10)$ 这类成比例的解。每个合法三角形都能按较小直角边、较大直角边、斜边重新排列成循环允许的形式，所以限制 $a \le b$ 不会漏掉不同形状或不同尺度的合法解。

5.13.10　例题十：三个人都只猜对一半

A、B、C、D 四支队伍比赛，最终分别获得第 1 到第 4 名，没有并列。赛前，甲预测“A 第一，B 第二”，乙预测“C 第一，D 第三”，丙预测“D 第二，A 第三”。已知每个人的两项预测恰好一项正确，求四支队伍的实际名次。用 `a`、`b`、`c`、`d` 分别保存 A、B、C、D 的名次，每个变量都应取 1 到 4 中的一个值。

甲的两项预测分别对应 `a == 1` 与 `b == 2`，每个比较的结果都是 0 或 1。“恰好一项正确”要求两个比较的结果不同，因此可写成 `(a == 1) != (b == 2)`。乙、丙的预测也按同样方式转换为条件，再用 `&&` 要求三个人的条件同时成立。

四个名次是 1、2、3、4，和为 10。只要从这四个数中选出互不相同的 a,b,c，剩下

的那个名次就必定是 $d = 10 - a - b - c$。因此只需要三重循环；`continue` 负责排除重复名次，`if` 负责检查三个人的预测。

```
#include <stdio.h>

int main(void) {
    for (int a = 1; a <= 4; a++) {
        for (int b = 1; b <= 4; b++) {
            if (b == a) {
                continue;
            }
            for (int c = 1; c <= 4; c++) {
                if (c == a || c == b) {
                    continue;
                }
                int d = 10 - a - b - c;
                if (((a == 1) != (b == 2)) &&
                    ((c == 1) != (d == 3)) &&
                    ((d == 2) != (a == 3))) {
                    printf("A=%d, B=%d, C=%d, D=%d\n", a,b,c,d);
                }
            }
        }
    }
    return 0;
}
```

唯一结果为 `A=3, B=2, C=1, D=4`。甲只猜中 B 第二，乙只猜中 C 第一，丙只猜中 A 第三。`b == a` 后的 `continue` 作用于枚举 `b` 的循环，直接尝试下一个 `b`；`c == a || c == b` 后的 `continue` 作用于枚举 `c` 的循环，直接尝试下一个 `c`。两处都只跳过所在循环的本轮剩余部分。

5.13.11　例题十一：方程的所有正整数解

求 $x + 2y^2 = 200$ 的全部正整数解。因为 $x \geq 1$，所以 $2y^2 \leq 199$，从而 $1 \leq y \leq 9$。确定 y 以后，$x = 200 - 2y^2$ 已经唯一确定，不必再枚举 x。

```c
#include <stdio.h>

int main(void) {
    for (int y = 1; y <= 9; y++) {
        int x = 200 - 2 * y * y;
        printf("x=%d, y=%d\n", x, y);
    }
    return 0;
}
```

九组 (x, y) 依次为 $(198, 1)$、$(192, 2)$、$(182, 3)$、$(168, 4)$、$(150, 5)$、$(128, 6)$、$(102, 7)$、$(72, 8)$、$(38, 9)$。$y = 10$ 给出 $x = 0$，不属于正整数解。

也可以用两重循环，让 x 从 1 到 199、y 从 1 到 10，再用 `if (x + 2 * y * y == 200)` 筛选。它能得到同一解集，但要检查 1990 组候选。先利用等式消去 x，只需处理 9 组。

5.14　习题与解析

习题一：最后一次失败的判断也会自增吗

分别求下列两个独立片段输出的 `count` 与 `i`：

```c
int i = 1, count = 0;
while (i++ < 4) {
    count++;
}
printf("%d %d\n", count, i);
```

```c
int i = 1, count = 0;
while (++i < 4) {
    count++;
}
printf("%d %d\n", count, i);
```

解析：第一个条件依次用旧值 1、2、3、4 比较，每次比较都伴随自增。前三次进入循环体，第四次失败后 `i` 仍增加到 5，输出 `3 5`。第二个条件先把 `i` 加到 2、3、4，再比较，只有前两次成功，输出 `2 4`。

习题二：输入失败时，哪个字符还没读走

对前面的整数求和程序，循环条件为 `scanf("%ld", &num) == 1`。输入 `12 -3x 7` 后，程序输出的和是多少？停止时下一个尚未读取的字符是什么？若输入是一个没有任何内容的文件，能否删去条件中的 `== 1`？

解析：前两次转换分别得到 12 和 -3，累计和为 9；第二次转换读走`-3`，停在 `x` 之前。第三次转换无法把 `x` 转换为整数，返回 0，循环结束。因此输出的和为 9，下一个未读字符仍是 `x`，后面的空格和 7 也都没有处理。输入为空文件时，第一次 `scanf` 返回 `EOF`。它是一个负值，在条件中会被当作真；若删去 `== 1`，循环体就会错误地使用尚未赋值的 `num`。

习题三：修正一个不会前进的循环

下面想输出 1 到 5 中的奇数，却不能按预期结束。指出问题并修改。

```
int i = 1;
while (i <= 5) {
    if (i % 2 == 0) {
        continue;
    }
    printf("%d ", i);
    i++;
}
```

解析：第一轮输出 1，并把 `i` 增加为 2。此后每轮都满足 `i % 2 == 0`，执行 `continue` 后直接回到 `while` 条件，跳过循环体末尾的 `i++`。`i` 始终是 2，条件始终为真。改用下面的 `for` 循环后，`continue` 会先转到更新位置执行 `i++`，再判断是否继续，因此能依次输出 1、3、5 并结束：

```
for (int i = 1; i <= 5; i++) {
    if (i % 2 == 0) {
        continue;
    }
    printf("%d ", i);
}
```

也可以保持 `while`，仅当 `i % 2 != 0` 时输出，把 `i++` 放在分支之后，确保每轮都会更新。

习题四：三次判断中，谁的副作用会发生

设 x、y、result 都是 int。每个片段都从 x = 0, y = 3 重新开始，求 result 及最后的 x,y：

```
x = 0; y = 3;
result = (x != 0 && y++ > 2);

x = 0; y = 3;
result = (x == 0 || y++ > 2);

x = 0; y = 3;
result = (x++ == 0 && y++ > 2);
```

解析：第一段中，x != 0 为假，逻辑与的右操作数不求值，所以 y++ 不执行。最后 result=0，x=0,y=3。

第二段中,x == 0 为真,逻辑或的右操作数不求值,y++ 同样不执行。最后 result=1，x=0,y=3。

第三段中，x++ == 0 用 x 的旧值 0 比较，结果为真，并使 x 增加到 1。因为逻辑与的左操作数为真，还要计算右侧：y++ > 2 用旧值 3 比较，也为真，并使 y 增加到 4。因此最后 result=1，x=1,y=4。

习题五：break 跳出了哪里

预测完整程序的输出，并说明两处 break 分别作用于什么。

```c
#include <stdio.h>

int main(void) {
    for (int i = 0; i < 3; i++) {
        switch (i) {
            case 0: printf("A"); break;
            case 1: printf("B"); /* fall through */
            default: printf("C"); break;
        }
        printf("D");
    }
    putchar('\n');
    return 0;
}
```

解析：输出 ADBCDCD。i=0 打印 A 后离开 switch，再打印 D；i=1 打印 B 后贯穿到 default 打印 C，再打印 D；i=2 直接进入 default 打印 C，再打印 D。两处 break 都只结束 switch。

习题六：第一轮是否一定执行

令 n=0，count=0，分别执行 while (n > 0) { count++; } 与 do { count++; } while (n > 0);，count 各是多少？

解析：while 先判断 n > 0，条件为假，循环体一次也不执行，因此 count 仍为 0。do-while 先执行 count++，使 count 变为 1，再判断同一个条件并退出。因此两个片段中 count 的最终值分别是 0 和 1；差别来自第一次判断发生在循环体之前还是之后。

习题七：二维输出究竟有几次

在前面的字母三角形程序中，外层变量 row 仍从 0 取到 5。把内层初始化改为 ch = 'A'，循环条件改为 ch <= 'A' + row，每轮仍执行 ch++，每行结束后仍换行。程序会打印什么？一共打印多少个字母？仍假设使用 ASCII 字符编码。

解析：六行依次输出 A、AB、ABC、ABCD、ABCDE 和 ABCDEF。每开始一轮外层循环，内层的 ch 都重新取'A'，所以每行都从 A 开始。row 决定这一行的最后一个字母，各行字母数依次为 1 到 6，总数为 $1+2+3+4+5+6=21$，不包括换行。

习题八：列约数与判素数需要同样久吗

如果只要求回答 36 是否为素数，能否在发现约数 2 时 `break`？原来要求列出所有非平凡约数的程序能否这样改？

解析：2 是 36 除 1 和自身以外的正约数，发现它就足以判定 36 不是素数。因此，若任务仅是判素数，可以先执行 `is_prime = 0;`，再用 `break` 结束枚举 `divisor` 的内层循环。

若任务是列出全部非平凡正约数，输出 2 和 18 后还不能结束。继续尝试 3，会找到 3 和 12；尝试 4，会找到 4 和 9；尝试 5 没有新约数；尝试 6 则找到 6。此时若在发现 2 后立即退出，就会漏掉后面这些约数。能否提前结束循环，取决于已经得到的信息是否足以完成题目要求。

习题九：为边界选择输入

分别为成绩统计、油漆桶数和二次方程程序选择能区分不同分支的输入，并说明要观察什么。

解析：成绩统计可输入 `0 100 -1 101 q`，检查 0 和 100 被接收，而 -1 和 101 被拒绝。再分别输入`-1 101 q` 和 `q`，检查只有无效成绩、完全没有成绩这两种情况都不会计算平均分或输出极值的初始值。

油漆桶数可依次输入 0、1、199、200、201 和一个负数。前五项应分别得到 0、1、1、1、2 桶，负数应得到无效面积提示。这些值检查了面积为 0、尚不满一桶、恰好一桶和刚刚超过一桶的情况。

方程程序可分别输入 `1 -3 2`、`1 2 1`、`1 -2 5`、`0 2 -6`、`0 0 0`、`0 0 1`，依次检查两实根、重根、复根、一次方程、任意数都是解以及无解六条路径。还可输入 `1 q`，使第二个系数读取失败，检查程序是否立即结束，而没有使用尚未赋值的系数继续计算。

5.15　检查循环与分支

温度统计中的两个计数器，只有放在具体的执行位置上才有明确含义：读入一个温度但尚未执行 `all_days++` 时，总天数还不包含这条记录；完成本轮循环体后，两个计数器才反映所有已处理记录。循环负责继续读取，分支负责决定是否计入低温天数，循环结束后的判断则保证只有存在记录时才计算百分比。

检查其他循环与分支程序时，也可以沿着执行过程确认：

- 循环开始前，用于判断和计算的变量是否已有值。
- 每轮读取了什么输入；读取失败后会执行哪条路径。
- 哪些分支会跳过更新；跳过后还能否到达退出条件。
- 循环结束时，变量保存的是最后处理的值，还是下一个候选值。